



### 1. Problem Title & Link

**Title:** LeetCode 54 : Spiral Matrix

**Link:** [LeetCode 54 - Spiral Matrix](#)

### 2. Problem Statement (Short Summary)

You are given an  $m \times n$  matrix.

Return all elements of the matrix in **spiral order**.

Spiral order means:

- Traverse left  $\rightarrow$  right
- Then top  $\rightarrow$  bottom
- Then right  $\rightarrow$  left
- Then bottom  $\rightarrow$  top
- Continue until all elements are visited

### 3. Examples (Input $\rightarrow$ Output)

#### Example 1

Input:

matrix=

```
[  
  [1,2,3],  
  [4,5,6],  
  [7,8,9]  
]
```

Output:

[1,2,3,6,9,8,7,4,5]

Explanation:

Spiral traversal:

1  $\rightarrow$  2  $\rightarrow$  3

$\downarrow$

4 5 6

$\uparrow$       $\downarrow$

7  $\leftarrow$  8  $\leftarrow$  9

#### Example 2

Input:

matrix=

```
[
```



```
[1,2,3,4],  
[5,6,7,8],  
[9,10,11,12]  
]
```

Output:

```
[1,2,3,4,8,12,11,10,9,5,6,7]
```

### Example 3

Input:

```
matrix=[[1]]
```

Output:

```
[1]
```

## 4. Constraints

$1 \leq m, n \leq 10$

$-100 \leq \text{matrix}[i][j] \leq 100$

Special restrictions:

- Must visit each element exactly once
- Avoid revisiting cells

## 5. Core Concept (Pattern / Topic)

### Matrix Traversal + Simulation

## 6. Thought Process (Step-by-Step Explanation)

### Brute Force

Use another matrix:

- Mark visited cells
- Move in directions
- Change direction whenever boundary reached

Time Complexity:

$O(m*n)$

Space Complexity:

$O(m*n)$

Extra space is unnecessary.

### Optimized Thinking



Instead of tracking visited cells:

Maintain four boundaries:

top

bottom

left

right

Process in order:

1. Traverse top row
2. Traverse right column
3. Traverse bottom row
4. Traverse left column

After each traversal:

- Shrink corresponding boundary

Continue until:

top <= bottom

and

left <= right

## 7. Visual / Intuition Diagram (ASCII Diagram)

Input:

1 2 3

4 5 6

7 8 9

Iteration 1:

→ → →

↓

4 5 ↓

↑ ← ← ←

Visited:

1 2 3 6 9 8 7 4

Remaining:

5

Final output:

1 2 3 6 9 8 7 4 5

## 8. Pseudocode (Language Independent)

top=0



```
bottom=rows-1
```

```
left=0
```

```
right=cols-1
```

```
while top<=bottom and left<=right:
```

```
    traverse left→right
```

```
    top++
```

```
    traverse top→bottom
```

```
    right--
```

```
    if valid:
```

```
        traverse right→left
```

```
        bottom--
```

```
    if valid:
```

```
        traverse bottom→top
```

```
        left--
```

```
return result
```

## 9. Code Implementation

### Python

```
class Solution:
```

```
    def spiralOrder(self, matrix):
```

```
        result=[]
```

```
        top=0
```

```
        bottom=len(matrix)-1
```

```
        left=0
```

```
        right=len(matrix[0])-1
```

```
        while top<=bottom and left<=right:
```



```
for col in range(left,right+1):
    result.append(matrix[top][col])

top+=1

for row in range(top,bottom+1):
    result.append(matrix[row][right])

right-=1

if top<=bottom:

    for col in range(right,left-1,-1):
        result.append(matrix[bottom][col])

    bottom-=1

if left<=right:

    for row in range(bottom,top-1,-1):
        result.append(matrix[row][left])

    left+=1

return result
```

**Java**

```
class Solution {

    public List<Integer> spiralOrder(int[][] matrix) {

        List<Integer> result=new ArrayList<>();
```



```
int top=0;
int bottom=matrix.length-1;

int left=0;
int right=matrix[0].length-1;

while(top<=bottom && left<=right){

    for(int col=left;col<=right;col++)
        result.add(matrix[top][col]);

    top++;

    for(int row=top;row<=bottom;row++)
        result.add(matrix[row][right]);

    right--;

    if(top<=bottom){

        for(int col=right;col>=left;col--)
            result.add(matrix[bottom][col]);

        bottom--;
    }

    if(left<=right){

        for(int row=bottom;row>=top;row--)
            result.add(matrix[row][left]);

        left++;
    }
}
```



```

        return result;
    }
}

```

## 10. Time & Space Complexity

Time Complexity:

$O(m*n)$

Reason:

Each element is visited exactly once.

Space Complexity:

$O(1)$

Extra algorithm space:

Only boundary variables used.

(Result array excluded from complexity analysis.)

## 11. Common Mistakes / Edge Cases

1. Forgetting boundary checks

Wrong:

top++

bottom--

left++

right--

without validation

This can duplicate elements.

2. Missing single row case

[1,2,3]

3. Missing single column case

[

[1],

[2],

[3]

]

4. Traversing already visited rows



## 12. Detailed Dry Run (Step-by-Step Table)

Input:

```
[
  [1,2,3],
  [4,5,6],
  [7,8,9]
]
```

| Step         | top | bottom | left | right | Output              |
|--------------|-----|--------|------|-------|---------------------|
| Initial      | 0   | 2      | 0    | 2     | []                  |
| Top row      | 1   | 2      | 0    | 2     | [1,2,3]             |
| Right column | 1   | 2      | 0    | 1     | [1,2,3,6,9]         |
| Bottom row   | 1   | 1      | 0    | 1     | [1,2,3,6,9,8,7]     |
| Left column  | 1   | 1      | 1    | 1     | [1,2,3,6,9,8,7,4]   |
| Top row      | 2   | 1      | 1    | 1     | [1,2,3,6,9,8,7,4,5] |

Final:

[1,2,3,6,9,8,7,4,5]

## 13. Common Use Cases (Real-Life / Interview)

Image processing:

- Traverse pixels layer by layer

Robot movement:

- Spiral navigation path

Game development:

- Grid traversal

Matrix operations:

- Layer-by-layer processing

## 14. Common Traps (Important!)

1. Infinite loops due to wrong boundary updates
2. Missing:

top<=bottom

left<=right

checks

3. Visiting same row twice





4. Incorrect row/column indexes

## 15. Builds To (Related LeetCode Problems)

Progression:

- LC 54 → Spiral Matrix
- LC 59 → Spiral Matrix II
- LC 48 → Rotate Image
- LC 73 → Set Matrix Zeroes
- LC 240 → Search a 2D Matrix II

## 16. Alternate Approaches + Comparison

| Approach             | Time     | Space    | Notes                 |
|----------------------|----------|----------|-----------------------|
| Visited Matrix       | $O(m*n)$ | $O(m*n)$ | Easy but extra memory |
| Direction Simulation | $O(m*n)$ | $O(m*n)$ | Uses visited cells    |
| Boundary Traversal   | $O(m*n)$ | $O(1)$   | Optimal               |

## 17. Why This Solution Works (Short Intuition)

The algorithm processes one outer layer at a time using four boundaries. After finishing a layer, boundaries shrink inward. Since every element belongs to exactly one layer, each element gets visited exactly once.

## 18. Variations / Follow-Up Questions

1. Generate a spiral matrix instead of traversing
2. Traverse in anti-clockwise spiral order
3. Start traversal from center
4. Spiral traversal in linked matrix
5. Traverse matrix diagonally
- 6.